

EEE 416 (January 2026)

Microprocessor and Embedded Systems Laboratory

Final Project Report

Section: C2 Group: 04

Design and Implementation of a Bidirectional Bangla Braille Translation System

Course Instructors:

Tanushri Medha Kundu, Lecturer

Abrar Assaeem Fuad, Lecturer

Signature of Instructor: _____

Academic Honesty Statement:

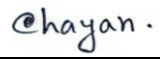
IMPORTANT! Please carefully read and sign the Academic Honesty Statement, below. Type the student ID and name, and put your signature. You will not receive credit for this project experiment unless this statement is signed in the presence of your lab instructor.

"In signing this statement, We hereby certify that the work on this project is our own and that we have not copied the work of any other students (past or present), and cited all relevant sources while completing this project. We understand that if we fail to honor this agreement, We will each receive a score of ZERO for this project and be subject to failure of this course."

Signature: 

Full Name: Mushfiquir Rahman

Student ID: 2106181

Signature: 

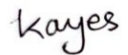
Full Name: Chayan Paul

Student ID: 2106182

Signature: 

Full Name: Shamin Yeaser

Student ID: 2106183

Signature: 

Full Name: Kayes Sami Malitha

Student ID: 2106184

Table of Contents

1	Abstract	1
2	Introduction	1
3	Design	3
3.1	Problem Formulation (PO(b))	3
3.1.1	Identification of Scope	3
3.1.2	Literature Review.....	3
3.1.3	Formulation of Problem.....	4
3.1.4	Analysis	4
3.2	Design Method (PO(a))	5
3.2.1	System Architecture and Component Selection.....	5
3.2.2	Wi-Fi Access Point and Web Server Hosting.....	5
3.2.3	Receiving and Cleaning the Input.....	6
3.2.4	Input Text Validation.....	6
3.2.5	Rendering Bangla Text on the OLED.....	7
3.2.6	Audio Subsystem	7
3.2.7	Audio Feedback Workflow	8
3.2.8	Solenoid Driver Stage	8
3.2.9	Pushbutton Input Subsystem.....	9
3.2.10	Complete Wiring of the Device	10
3.2.11	Power Budget and Supply Design	10
3.2.12	Braille Input Rules (Braille → Bangla)	10
3.2.13	Braille Output Pipeline (Bangla →Braille).....	13
3.3	Circuit Diagram	15
3.4	Simulation Model	15
3.5	CAD/Hardware Design	15
3.6	Veroboard Implementation	16
4	Implementation	17
4.1	Description.....	17
5	Design Analysis and Evaluation	22
5.1	Novelty	22
5.2	Design Considerations (PO(c))	23
5.2.1	Considerations for public health and safety	23

5.2.2	Considerations to the environment	23
5.2.3	Considerations to cultural and societal needs	23
5.3	Limitations of Tools (PO(e))	23
5.4	Impact Assessment (PO(f)).....	24
5.4.1	Assessment of Societal and Cultural Issues	24
5.4.2	Assessment of Health and Safety Issues	24
5.4.3	Assessment of Legal Issues	24
5.5	Sustainability Evaluation (PO(g)).....	24
5.6	Ethical Issues (PO(h))	25
6	Reflection on Individual and Team work (PO(i))	26
6.1	Individual Contribution of Each Member	26
6.2	Mode of Teamwork.....	26
6.3	Diversity Statement of Team	27
6.4	Log Book of Project Implementation.....	27
7	Communication to External Stakeholders (PO(j))	28
7.1	Executive Summary	28
7.2	User Manual.....	30
7.3	Github Link.....	31
7.4	YouTube Link.....	31
8	Project Management and Cost Analysis (PO(k)).....	32
8.1	Bill of Material.....	32
8.2	Calculation of Per Unit Cost of Prototype	33
8.3	Calculation of Per Unit Cost of Mass-Produced Unit.....	33
9	Future Work (PO(l))	34
10	References.....	34

1 Abstract

Braille is the primary reading and writing medium for people who are blind, but very few sighted family members, teachers or classmates can read it, so communication between the two groups breaks down in both directions. Commercial refreshable Braille displays are built around English and cost several hundred US dollars, which places them far beyond the reach of most Bangladeshi learners. This project presents the design and implementation of a low-cost, bidirectional Bangla Braille translation device built around an ESP32 microcontroller. In the WRITE direction, the user types on a six-dot Braille keyboard read through a PCF8575 I²C expander; the firmware decodes the dot pattern into the corresponding Bangla character, renders it on a 128 × 64 OLED display using the BanglaText and U8g2 libraries, and plays the matching pre-recorded clip through a DFPlayer Mini and a 4 Ω speaker. In the READ direction, Bangla text is sent from any phone or computer to a web page hosted on the ESP32's own Wi-Fi access point; the firmware decodes the UTF-8 stream, applies the Bangla Braille rules for কার signs, conjuncts, ঞ and ঙ, numbers and punctuation, and drives six solenoids that form one refreshable tactile Braille cell, which the reader steps through with NEXT and PREVIOUS buttons. Content and cursor position are preserved when the user toggles between the two modes. The complete prototype was realised on veroboard at a material cost of about BDT 10,840, roughly one-fifth the price of the cheapest commercial refreshable display.

2 Introduction

Braille is a six-dot tactile code that gives blind readers direct, independent access to written language. In Bangladesh it is taught in integrated schools and used in examinations, yet almost nobody outside the blind community can read it. The result is a two-way communication gap: a blind student can write a note in Braille that a sighted teacher cannot read, and a sighted parent can write a note in Bangla that a blind child cannot read. Closing that gap requires a device that translates in both directions, not merely a reader or a writer.

Devices that could close part of this gap already exist, but they are not practical here. Refreshable Braille displays such as the HumanWare Brailliant BI 40X or the Orbit Reader 20 are engineered around English contracted Braille and are priced from about US\$ 449 upwards [1], [2] — several times the monthly income of a typical Bangladeshi household. None of them implement the Bangla Braille code, which has its own rules for dependent vowels (কার), conjunct clusters (যুক্তাক্ষর), the special letters ঞ and ঙ, and number and punctuation signs [5], [6].



Figure 1: (Left) Embossed Braille text, the medium blind readers use [3]. (Right) A commercial refreshable Braille display, HumanWare Brailliant BI 40X, priced beyond most Bangladeshi users [4].

The objective of this project is therefore a single, low-cost, self-contained device that performs both conversions: Bangla Braille input to Bangla text with speech output, and Bangla text input to physical, refreshable Braille output. Figure 2 shows the finished prototype and the two directions of translation

it supports.

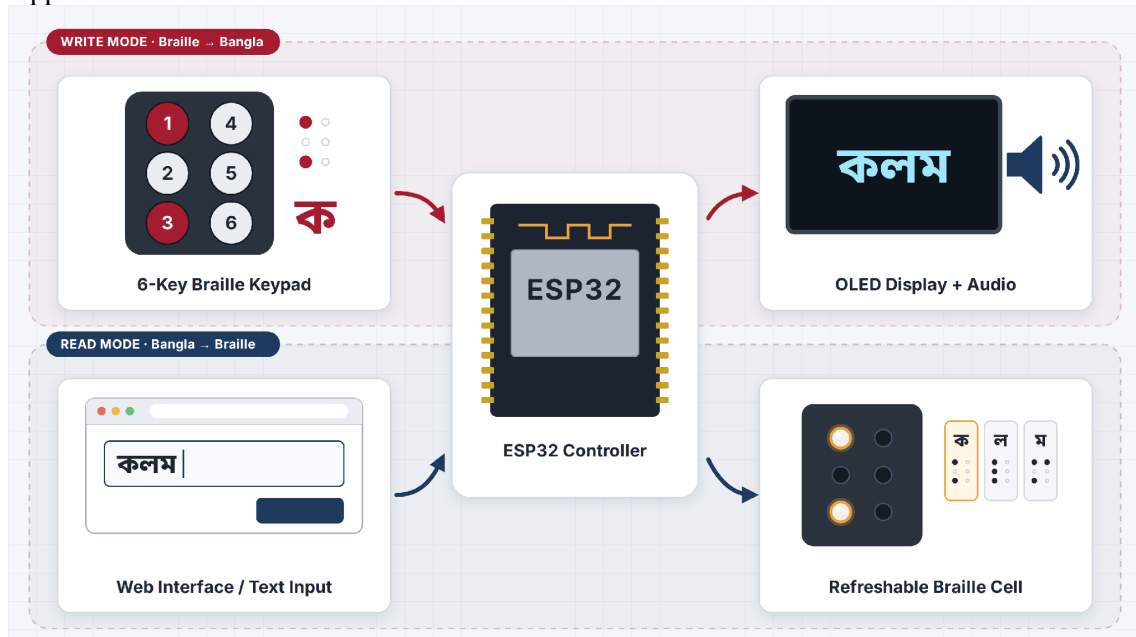


Figure 2: Concept of the bidirectional Bangla Braille communication system: Braille keyboard → Bangla text and speech, and Bangla text → refreshable tactile Braille.

Why this is a complex engineering problem

The problem satisfies several of the attributes of a complex engineering problem:

- Depth of knowledge required — the solution cannot be reached from one discipline alone. It needs embedded firmware, electromagnetic actuator driving, power electronics, I²C and UART peripheral interfacing, Unicode text processing, web technology and a working knowledge of the Bangla Braille standard.
- Conflicting requirements — tactile output demands high coil current and mechanical force, while portability demands low power and low heat. Low cost conflicts with the number of Braille cells; responsiveness conflicts with the debouncing needed by mechanical latching switches.
- No obvious solution — Bangla Braille is not a one-to-one character mapping. A single Bangla grapheme may occupy one, two or three cells, and one cell (for example dots 2-5-6) means different things inside and outside number mode, so the translation is context dependent and must be resolved by an ordered rule set rather than a lookup table alone.
- Infrequently encountered issues — silent অ, zero-width joiners in য় versus রা, and the two-cell forms of ঞ and ঳ have no equivalent in English Braille and are not handled by any off-the-shelf library.
- Outside the scope of standards — no Bangladeshi standard covers a single-cell refreshable display, so the cell geometry, the READ/WRITE interaction model and the navigation scheme all had to be defined by the team.
- Diverse groups of stakeholders — blind users, sighted family members and teachers, and Braille instructors all interact with the same device through different modalities (touch, sight and hearing).
- Interdependence — the keyboard, decoder, display, audio player, translator and solenoid driver are separate subsystems, but a change in any one of them (for example the shared I²C bus or the 5 V rail) propagates through the rest.

Alternative solutions considered

Three broad approaches were weighed before the present architecture was adopted:

- A smartphone application with a camera — the phone photographs embossed Braille and reads it aloud, as in the geometric approach of Kamal et al. [7]. This is inexpensive but works in one direction only, needs good lighting and a steady hand, and gives the blind user no tactile output at all.
- A full multi-cell refreshable display — 20 or 40 piezo cells, as used commercially [2]. This gives the best reading experience but the piezo bimorph stack alone costs more than the entire budget of this project.
- A single refreshable cell with solenoid actuation plus audio and visual feedback — the option chosen here. One cell keeps the cost and the power budget within reach, the loss in reading speed is compensated by the spoken feedback, and the same microcontroller can host both translation directions and the web interface.

3 Design

3.1 Problem Formulation (PO(b))

3.1.1 Identification of Scope

The device was scoped around the following functions, all of which are implemented in the delivered prototype:

- Designing a six-dot physical Braille keyboard with latching pushbuttons.
- Converting an entered Braille pattern into the corresponding Bangla character.
- Supporting Bangla vowels, consonants, কার signs, conjunct clusters, numbers, punctuation and the special Braille sequences for ঞ and ঙ.
- Displaying the generated Bangla text on an OLED and providing spoken audio feedback for every entry.
- Receiving Bangla text from any phone or computer through a web page hosted by the ESP32 itself.
- Converting that Bangla text into six-dot Braille patterns and producing physical Braille output through six solenoids.
- Allowing the reader to navigate the translated message cell by cell with NEXT and PREVIOUS controls, and to switch between READ and WRITE modes without losing the current content or position.

Deliberately left outside the scope are: grade-2 (contracted) Braille, languages other than Bangla, synthesised text-to-speech, multi-cell displays and an enclosure certified for commercial sale. These are revisited in Section 9.

3.1.2 Literature Review

Existing work was surveyed along two lines: the definition of the Bangla Braille code itself, and hardware that reads or writes Braille. Table 1 summarises the sources that shaped this design and the gap each one leaves.

Existing work	What it does	Gap for our problem
Rahman, Rahman & Kaykobad — Bangla Braille Adaptation [5]	Defines the Bangla Braille code: letters, কৱ, conjunct and number rules	A standard only — no device
World Braille Usage, 3rd ed. — Bangladesh chart [6]	Official Bangla Braille cell for every letter, কৱ, sign, digit and punctuation mark	Reference chart only — no device
Kamal et al. — Braille-to-text for Bengali [7]	Scanned embossed Braille → Bangla text (97.25 % accuracy)	One direction; needs a scanner; no tactile output
Tabassum et al. — low-cost mechanical Braille display [8]	Refreshable tactile display hardware for Bangladeshi students	Output only; no Braille typing or text entry
Orbit Reader 20 (commercial) [2]	20-cell refreshable display with Bluetooth / USB	US\$ 449; not built around Bangla rules
Arduino refreshable Braille cell project [9]	Single do-it-yourself cell driven by a microcontroller	English letters, one direction only
This project	Braille ⇌ Bangla in one device, with OLED, audio feedback and web input	Handles কৱ, silent অ, conjuncts, ঞ / ৭ and numbers

Table 1: Survey of existing work and the gap each leaves for this project.

The survey shows a clear vacancy. The linguistic rules are fully documented [5], [6] and low-cost tactile hardware has been demonstrated [8], [9], but no reported system combines the two directions for Bangla in a single affordable unit. That vacancy defines the problem addressed here.

3.1.3 Formulation of Problem

The overall task separates into two conversion problems that share the same hardware and the same microcontroller, plus a state-management problem that links them.

Problem 1 — Braille to Bangla (WRITE mode).

Workflow: six Braille keys → Braille pattern → Bangla character → OLED + audio. Formally, the firmware must realise a mapping

$$f: \{0,1\}^6 \times S \rightarrow \Sigma \times S \quad (3.1)$$

where $\{0,1\}^6$ is the set of 64 possible dot patterns of one cell, Σ is the Bangla character set and S is the decoder state (pending conjunct prefix, pending dot-5 prefix, number mode, last emitted character). The state argument is unavoidable: the same pattern produces different output depending on what preceded it.

Problem 2 — Bangla to Braille (READ mode).

Workflow: Bangla text → UTF-8 processing → Braille pattern → solenoid output. The firmware must realise the inverse mapping

$$g: \Sigma^* \rightarrow (\{0,1\}^6)^* \quad (3.2)$$

which is not injective in either direction and is not length preserving: one Bangla grapheme may expand to one, two or three cells (a conjunct cluster expands to a prefix cell plus one cell per consonant), and conversely a prefix cell carries no character of its own.

Mode switching.

A single latching MODE button toggles the two problems. The requirement is that the text buffer of WRITE mode and the cell index of READ mode both survive the toggle, so that a user can feel a cell, switch to WRITE to answer, and switch back without losing position.

3.1.4 Analysis

Each major design decision was taken by comparing the realistic alternatives against the project constraints: a material budget under BDT 8,000, operation from a single 5 V rail, no dependence on an internet connection or an external router, and full coverage of the Bangla Braille code. Table 2 records the comparison.

Decision	Options considered	Chosen option and reason
Tactile actuator	Piezoelectric cell, micro servo, solenoid	Solenoid — low cost, fast, and driven by a simple on/off signal; piezo cells cost more than the whole budget and servos are bulky and noisy
Solenoid driver	Direct GPIO, bipolar transistor, logic-level MOSFET	IRLB8721 MOSFET — fully enhanced at 3.3 V gate drive with very low conduction loss; a flyback diode protects it [10]
Button inputs	Nine direct GPIO pins, I/O expander	PCF8575 — all nine buttons on the two-wire I ² C bus already used by the OLED, freeing ESP32 pins for the six solenoid channels [10]
Audio feedback	Cloud text-to-speech, pre-recorded clips	DFPlayer Mini with microSD — instant playback and fully offline operation [10]
Bangla text input	File upload to flash, live web form	ESP32-hosted web page — usable from any phone or PC with no app and no router
Bangla display	Hand-made bitmaps, font library	BanglaText with U8g2 — shapes ଫ signs and conjuncts automatically [11], [12]

Table 2: Design decisions, alternatives considered and the reason for each choice.

3.2 Design Method (PO(a))

This section sets out how knowledge of mathematics, electronics and computer engineering was applied to realise the two mappings of equations (3.1) and (3.2). It moves from the hardware architecture, through the input, display and audio subsystems, to the power electronics, and then to the two translation algorithms.

3.2.1 System Architecture and Component Selection

The system is built around a 30-pin ESP32 DevKit, which carries all the logic, hosts the Wi-Fi access point and runs the web server. Table 3 lists the components and the role of each in the system.

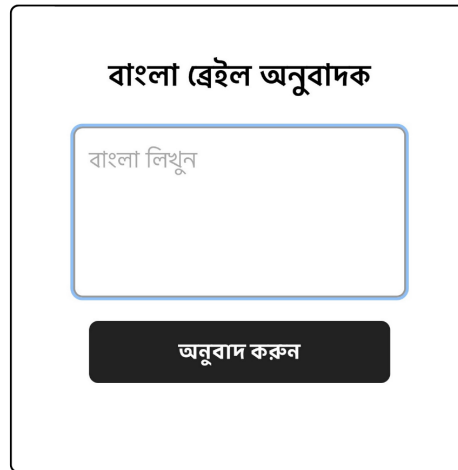
Component	Role in the system
ESP32 DevKit (30-pin)	All logic, Wi-Fi access point and web server
SH1106 128 × 64 OLED display (I ² C)	Shows Bangla text in both modes
PCF8575 I/O expander	Reads the six dot keys plus ENTER/NEXT, BACKSPACE/PREVIOUS and MODE
DFPlayer Mini + microSD + speaker	Plays 104 pre-recorded Bangla clips
5 V 1 A push-pull solenoids × 6	Form one refreshable tactile Braille cell
IRLB8721 MOSFET + 1N5819 diode × 6	Logic-level switching and flyback protection for each coil
LiPo battery and buck converter	5 V supply for the solenoids and the DFPlayer

Table 3: Hardware components and their role in the system.

Two serial buses carry all peripheral traffic. The OLED (address 0x3C) and the PCF8575 (address 0x20) share one I²C bus on GPIO21 (SDA) and GPIO22 (SCL); the DFPlayer Mini uses a UART on GPIO17 (TX) and GPIO16 (RX). The six solenoid channels take GPIO25, 33, 32, 14, 27 and 26. Sharing the I²C bus is what makes six independent solenoid outputs possible on a 30-pin board.

3.2.2 Wi-Fi Access Point and Web Server Hosting

The ESP32 creates its own Wi-Fi access point, so no external router and no internet connection is required. The network is named Braille_Bangla with the password 123456789, and once a phone or laptop joins it the device is reached at 192.168.4.1. The HTML of the page is stored in the firmware itself, in the header file WebPage.h; the root route “/” serves that page and the route “/send” receives the text when the user presses the send button.

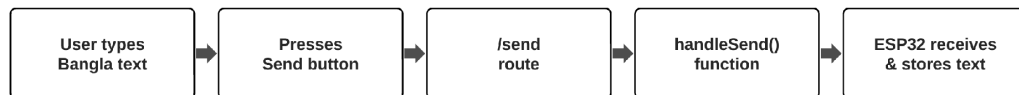


The ESP32-hosted webpage — বাংলা ব্রেইল অনুবাদক

Figure 3: The ESP32-hosted web page বাংলা ব্রেইল অনুবাদক, through which a sighted user types Bangla text.

3.2.3 Receiving and Cleaning the Input

The text posted to “/send” is handled by `handleSend()`. Before the text is used it is cleaned: carriage-return and line-feed characters are converted into single spaces, and leading and trailing spaces are stripped. This step was added after an early version misbehaved when raw Enter characters reached the translator, because a newline inside the buffer terminated number mode in the wrong place and shifted every subsequent cell.



Example

Before cleaning

বাংলা
ব্রেইল



After cleaning

বাংলা ব্রেইল

handleSend() cleans the text before checking it for Bangla.

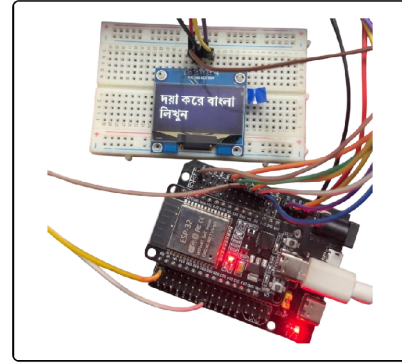
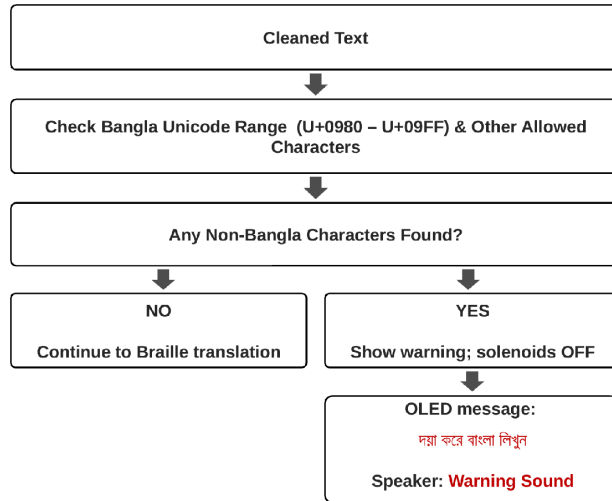
Input Cleaning Rules

- Enter / new-line characters → converted to spaces
- Extra spaces at the start and end → removed

Figure 4: Path taken by the text from the browser to the ESP32, and the effect of the cleaning rules.

3.2.4 Input Text Validation

The cleaned string is then validated. Every code point is tested against the Bengali Unicode block, U+0980 to U+09FF, together with a short list of additionally permitted characters (space, the Bangla দাঁড়ি and the punctuation marks that have Braille cells). If the string contains characters outside that set the translation is abandoned: the OLED shows দয়া করে বাংলা লিখুন, a warning clip is played, and all six solenoids are de-energised so that the reader is never left with a stale cell under the fingertip.



showing the Bangla-only warning message

Figure 5: Validation of the received text and the warning state shown on the OLED when non-Bangla input is detected.

3.2.5 Rendering Bangla Text on the OLED

Bangla cannot be drawn by a simple character-to-bitmap loop, because কার signs attach to the left, right, above or below the consonant they modify and conjuncts change glyph shape entirely. The BanglaText library by Mamunul Mazid [11] is therefore used on top of the U8g2 monochrome graphics library [12]. BanglaText carries a font header file that stores the Bangla glyphs together with their pixel mapping and performs the reordering and shaping; U8g2 pushes the resulting bitmap to the SH1106 panel over I²C.



Figure 6: Chain from Bangla text to OLED pixels through the BanglaText font library [11] and U8g2 [12].

3.2.6 Audio Subsystem

Audio feedback is produced by a DFPlayer Mini reading MP3 files from a microSD card [13] and driving a 4 Ω , 3 W speaker directly from its on-board amplifier. The module is commanded over UART, so a playback request costs the ESP32 only a short serial frame and no processing time. There are 104 clips on the card in total: one for each Bangla letter, কার sign and digit, and additional clips for space, new line, backspace, mode change and invalid input.

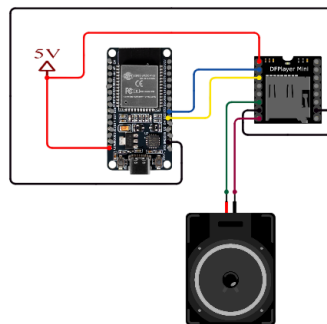


Figure 7: DFPlayer Mini module and speaker used for the spoken feedback [13].

3.2.7 Audio Feedback Workflow

In WRITE mode every confirmed entry is spoken back immediately, which is what allows a blind user to verify a keystroke without sighted help. The sequence is:

- The user latches a dot pattern on the six keys and presses ENTER.
- The pattern is decoded to a Bangla character through the state machine of equation (3.1).
- The ESP32 sends the track number associated with that character to the DFPlayer over UART.
- The DFPlayer reads the corresponding MP3 file from the microSD card — for example the clip for ঐ — and plays it on the speaker.

Because the mapping from character to track number is a fixed table, no speech synthesis is needed and the response is effectively instantaneous.

3.2.8 Solenoid Driver Stage

Each of the six dots is a 5 V push-pull solenoid drawing about 1 A when energised, far beyond the 40 mA that an ESP32 pin can source, so every dot has its own low-side switch. The stage is repeated identically six times and is shown in Figure 8.

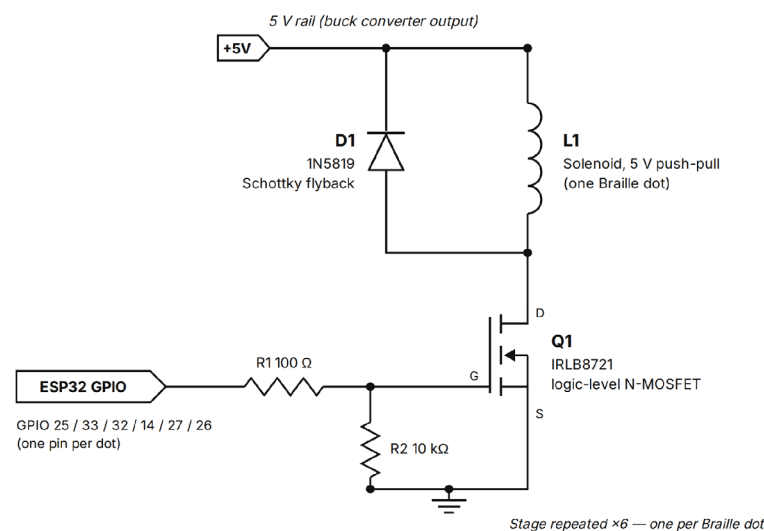


Figure 8: Solenoid driver stage for one Braille dot, repeated for all six dots.

The switching components were selected so that every rating is met with a wide margin, as set out in Table 4.

Component	Parameter	Device rating	Value in this design
IRLB8721 MOSFET	Maximum drain–source voltage V_{DS}	30 V	5 V rail, clamped to about 5.4 V at turn-off
IRLB8721 MOSFET	Maximum continuous drain current I_D	62 A	1 A per coil
IRLB8721 MOSFET	Gate threshold voltage $V_{GS(th)}$	2.5 V	3.3 V gate drive from the ESP32 GPIO
1N5819 Schottky diode	Reverse standoff voltage	40 V	5 V across the coil
1N5819 Schottky diode	Average forward current	1 A	1 A freewheeling coil current
Push-pull solenoid	Supply voltage and coil current	5 V, 1 A	5 V rail, about 1 A when energised

Table 4: Ratings of the switching components in one driver channel and the values they actually see.

The 30 V drain–source rating gives roughly a six-fold margin over the 5 V rail, and the 62 A drain-current rating is more than sixty times the 1 A coil current, so the MOSFET is operating far inside its safe area and no derating for temperature rise is necessary. The 2.5 V gate threshold is the figure that

makes the part usable here at all: it is below the 3.3 V that an ESP32 pin can deliver, which is why a logic-level device was specified rather than a standard MOSFET needing 10 V of gate drive. On the diode side, the 40 V reverse rating covers the clamped spike with a very large margin, while the 1 A forward rating matches the full coil current it must carry during the freewheel interval.

MOSFET switch.

The IRLB8721 is a logic-level N-channel MOSFET: it is fully enhanced by the 3.3 V that an ESP32 GPIO delivers, with a typical on-state resistance of about 16 mΩ. The conduction loss in each device is therefore

$$P_{cond} = I_D^2 \cdot R_{DS(on)} = (1\text{ A})^2 \times 0.016\ \Omega \approx 16\text{ mW} \quad (3.3)$$

which needs no heatsink. A 100 Ω series gate resistor limits the peak gate-charging current to

$$I_{G(peak)} = V_{GS} / R_I = 3.3\text{ V} / 100\ \Omega = 33\text{ mA} \quad (3.4)$$

which is comfortably inside the ESP32 pin rating, and a 10 kΩ pull-down holds the gate at 0 V while the ESP32 boots so that no dot is raised spuriously at power-up. The steady-state current wasted in that pull-down is only 3.3 V / 10 kΩ = 0.33 mA per channel.

Flyback protection.

A solenoid is an inductive load. When the MOSFET turns off, the collapsing field generates

$$v_L = L \cdot di/dt \quad (3.5)$$

and because di/dt is very large at turn-off this spike would exceed the drain-source rating of the MOSFET. A 1N5819 Schottky diode connected across the coil provides a freewheeling path, clamping the drain to roughly $V_+ + 0.4\text{ V}$. The Schottky type is chosen over an ordinary silicon diode for its low forward drop and fast recovery, so that the stored energy $\frac{1}{2} L \cdot I^2$ decays quickly enough for the cell to refresh at a usable rate.

3.2.9 Pushbutton Input Subsystem

Nine buttons are needed: six latching switches for dots 1 to 6, one latching switch for MODE, and two momentary switches whose function depends on the mode — ENTER/NEXT and BACKSPACE/PREVIOUS. Wiring all nine to the ESP32 directly would consume the pins reserved for the solenoids, so they are read instead through a PCF8575 16-bit I/O expander sitting on the same I²C bus as the display. The expander has internal pull-ups, so each switch simply grounds its port pin, and the whole keyboard is read in a single two-byte I²C transaction.

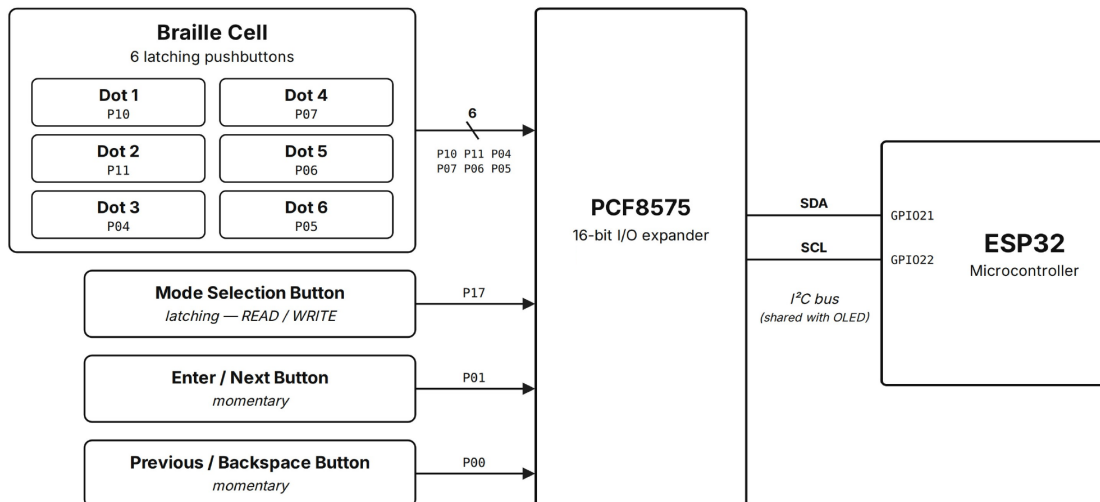


Figure 9: Pushbutton input subsystem: nine switches read through the PCF8575 expander on the shared I²C bus.

Latching switches were chosen for the six dots deliberately. A blind user can set several dots, feel the mechanical state of the keys to confirm the pattern, and only then press ENTER — an affordance that momentary buttons would not give. Their state is polled and software-debounced; the pattern is only committed on the ENTER edge.

3.2.10 Complete Wiring of the Device

Figure 10 brings the subsystems together. The ESP32 sits at the centre; the OLED and PCF8575 hang off the I²C bus on the left, the DFPlayer on the UART below, and the six MOSFET drivers and solenoids on the right. The solenoid rail and the ESP32 share a common ground, and the 5 V for the coils comes from the buck converter rather than from the ESP32 regulator, so that a coil inrush cannot brown out the microcontroller.

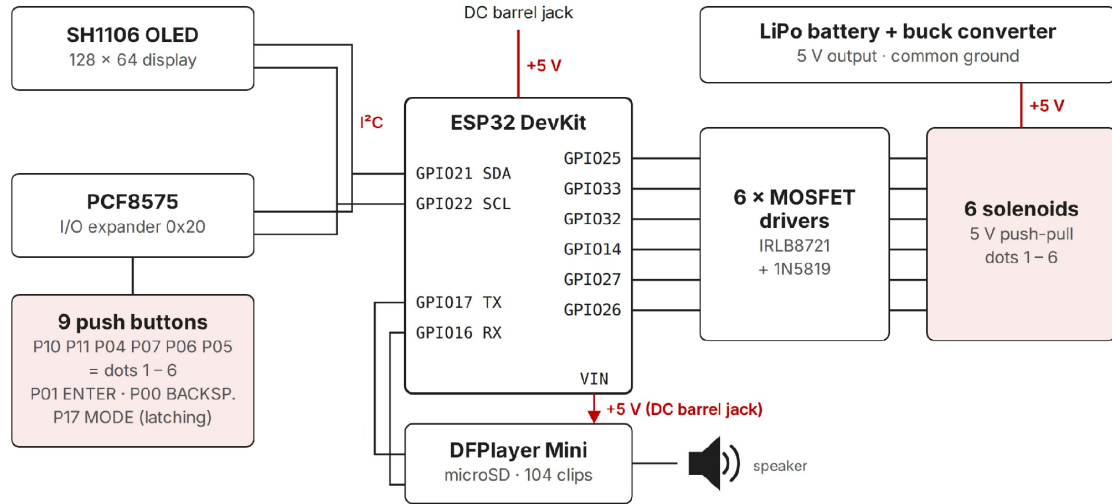


Figure 10: Complete wiring of the device, showing the two serial buses, the six solenoid channels and the power distribution.

3.2.11 Power Budget and Supply Design

The supply was sized for the worst case in which all six dots are raised simultaneously, the Wi-Fi access point is transmitting and a clip is playing. Each coil is a 5 V, 1 A device, so its winding resistance is

$$R_{\text{coil}} = V / I = 5 \text{ V} / 1 \text{ A} = 5 \Omega, \quad P_{\text{coil}} = V \cdot I = 5 \text{ W} \quad (3.6)$$

and the total demand is the sum in Table 5.

Load	Current (approximate)
6 solenoids $\times$ 1 A (worst case: all dots raised)	6.0 A
ESP32 with Wi-Fi access point (peaks)	0.5 A
DFPlayer Mini + 3 W speaker	0.6 A
OLED + PCF8575	$< 0.05 \text{ A}$
Total	$\approx 7.2 \text{ A} \rightarrow 36 \text{ W}$

Table 5: Worst-case power budget of the complete device.

Currents other than the solenoid current are typical datasheet-level figures; the purpose of the calculation is to fix the supply rating rather than to predict consumption precisely. A LiPo pack with a buck converter rated at 90 W was used, leaving a margin of well over two to one. In practice the average draw is far lower, because a Braille cell is only energised while the reader's finger is on it and a typical pattern raises two or three dots rather than six.

3.2.12 Braille Input Rules (Braille $\rightarrow$ Bangla)

A Braille cell has six dots, so there are $2^6 = 64$ distinguishable patterns, 63 of them non-blank. Bangla has far more graphemes than that, which is precisely why the code is context dependent and why the state argument appears in equation (3.1). Six cases had to be handled explicitly [5], [6]; each is described below with the corresponding cell diagrams.

Case 1 — Dependent vowels (কার).

In Braille an independent vowel (আ, ই) and its dependent form (া, ি) share exactly the same cell. The decoder therefore decides from context: if the vowel pattern arrives directly after a consonant, it is rendered as a কার-attached form of that consonant.

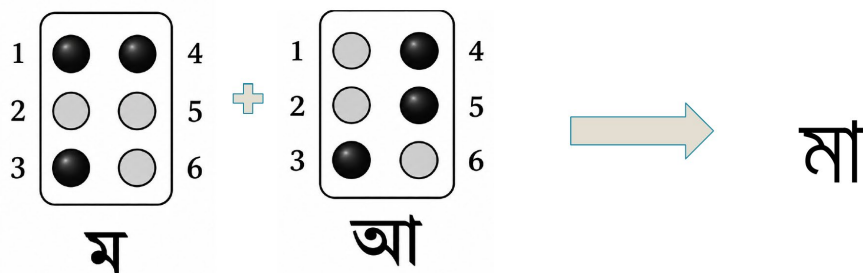


Figure 11: Case 1 — a consonant cell followed by a vowel cell produces the কার-attached form (ম + আ → মা).

Case 2 — Independent vowels (স্বরবর্ণ).

The same pattern must stay independent when it begins a word, and also when it follows a consonant that has been explicitly closed with the inherent অ sound. The second rule is what distinguishes বই from বি and is handled by tracking whether an explicit অ cell was entered.

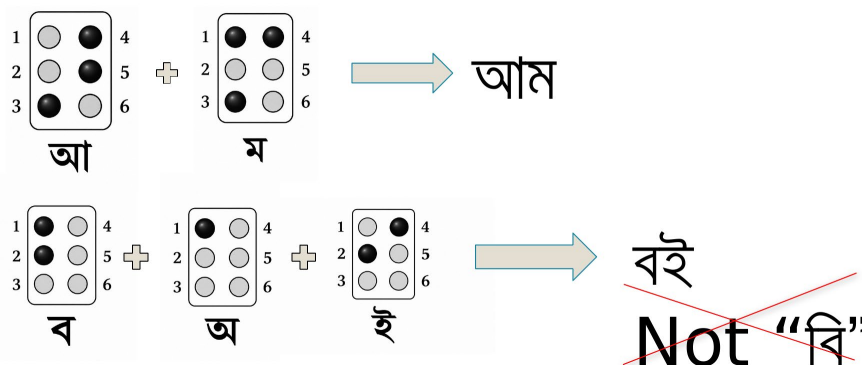


Figure 12: Case 2 — a vowel starting a word stays independent (আ + ম → আম); an explicit অ between the consonant and the vowel keeps বই from collapsing into বি.

Case 3 — Conjunct letters (যুক্তাক্ষর).

Bangla Braille does not mark a conjunct with a হসন্ত after the fact; it announces one in advance with a prefix cell. Dot 4 alone signals that the next two letters are joined, and dots 4 and 6 together signal that the next three are joined. The decoder therefore latches the prefix, collects the announced number of consonants and only then emits the cluster with the hasanta characters inserted.

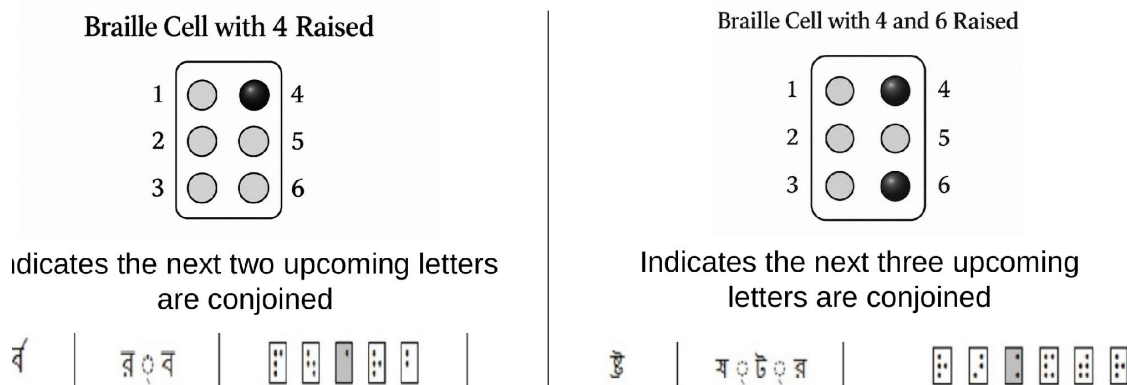
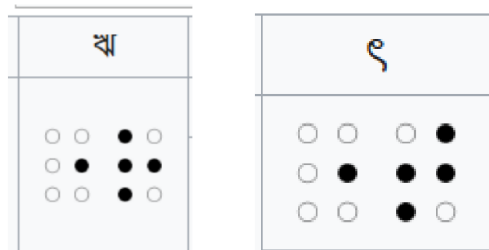


Figure 13: Case 3 — the conjunct prefix cells: dot 4 for a two-letter cluster and dots 4-6 for a three-letter cluster.

Case 4 — The special characters ঋ and ৙.

These two letters occupy two consecutive cells, but they follow a regular pattern: dot 5 followed by র gives ঋ, and dot 5 followed by ত gives ৙. With only one physical cell available, the decoder emulates the two-cell sequence in time: when the entered pattern is dot 5 alone it does not emit anything, but waits for the next entry and combines the two.



Dot 5+ র

Dot 5+ ত

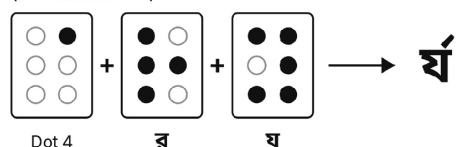
We only have one Braille cell, but we can render these two letters by waiting for two consecutive inputs if the first input is a Dot 5 only

Figure 14: Case 4 — the two-cell sequences for ঋ and ৙, emulated on a single cell by waiting for the entry that follows a lone dot 5.

Case 5 — Conjuncts with র (য and র্য).

Both forms begin with the dot-4 conjunct prefix. Pressing dot 4 once joins র and য with a hasanta and yields য, as in কার্য. Pressing dot 4 twice inserts a zero-width joiner before the hasanta and yields র্য, as in র্যাব. The two strings differ by a single invisible code point, U+200D, and would be indistinguishable on screen without this rule.

Dot 4 once — normal conjunct
(র + hasanta + য)



Dot 4 twice — ZWJ conjunct
(র + ZWJ + hasanta + য)

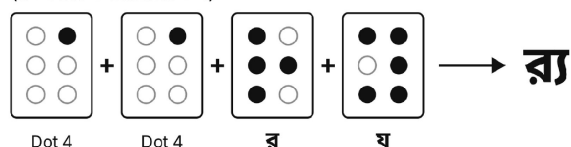
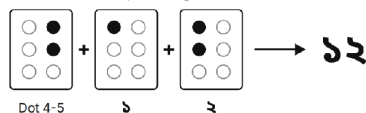


Figure 15: Case 5 — one dot-4 prefix gives the ordinary conjunct য; two dot-4 prefixes insert a zero-width joiner and give র্য.

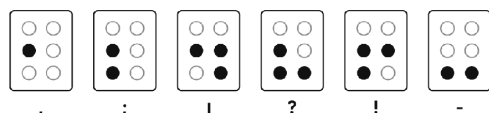
Case 6 — Numbers and punctuation.

A number begins with the number sign, dots 4-5; every cell after it is read as a digit until a space or a new line ends number mode. Punctuation marks have cells of their own. The interesting conflict is dots 2-5-6, which means দাঁড়ি (।) in running text but a decimal point inside a number: the sequence 4-5, 1-4, 2-5-6, 1 therefore reads ৩.১, whereas a space before the 2-5-6 cell ends number mode and gives ৩। instead.

Number: Dot 4-5, then digit cells



Punctuation cells



Dots 2-5-6 mean দাঁড়ি (।) in text but a decimal point (.) inside a number: 4-5, 1-4, 2-5-6, 1 → ৩.১. A space ends number mode, so space + 2-5-6 after a number gives ৩।

Figure 16: Case 6 — the number sign and digit cells, the punctuation cells, and the context-dependent meaning of dots 2-5-6.

3.2.13 Braille Output Pipeline (Bangla → Braille)

The reverse mapping of equation (3.2) is implemented as a four-stage pipeline that runs once, when a message arrives. Afterwards the NEXT and PREVIOUS buttons only move an index into the translated buffer; nothing is re-translated, so navigation is instantaneous and cannot drift out of step with the text.

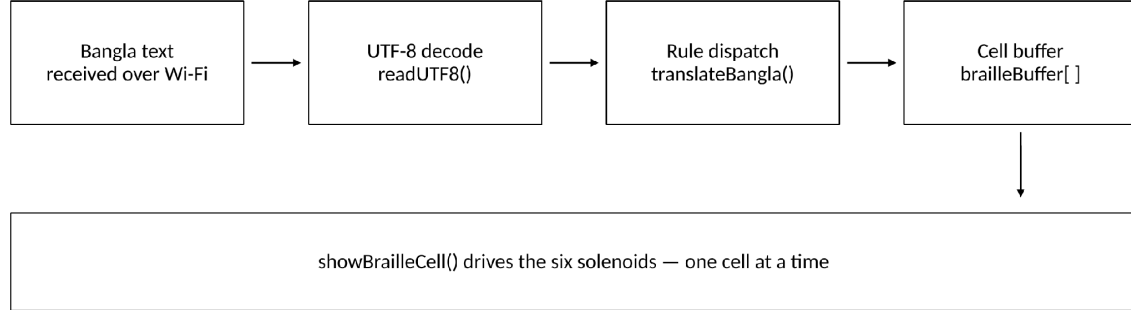


Figure 17: Translation pipeline from received Bangla text to the six solenoids.

Step 1 — Decoding the Bangla text.

Bangla code points lie in U+0980–U+09FF and are therefore three bytes each in UTF-8. The helper readUTF8() reconstructs the code point from the three bytes as

$$cp = ((b_0 \& 0x0F) \ll 12) \mid ((b_1 \& 0x3F) \ll 6) \mid (b_2 \& 0x3F) \quad (3.7)$$

For the bytes E0 A6 95 this gives 0x0995, which is ঐ. The decoded code point is then matched against the Braille lookup table.

One Bangla letter = three bytes

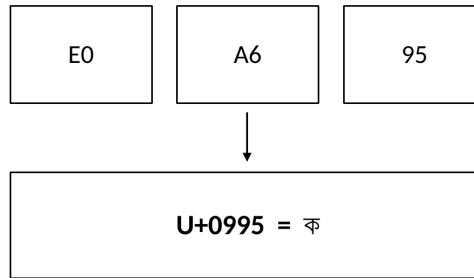


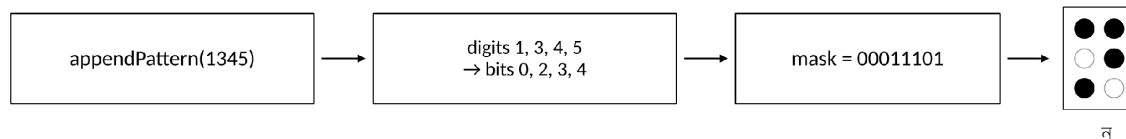
Figure 18: Step 1 — three UTF-8 bytes are reassembled into one Bangla code point.

Step 2 — Storing a Braille cell.

Each cell is stored as a single byte in brailleBuffer[], with bit 0 representing dot 1 up to bit 5 representing dot 6. The helper appendPattern() converts a human-readable dot list into that mask:

$$mask = \sum_{i=0}^5 2^i (d_i - 1) \quad (3.8)$$

so appendPattern(1345), meaning dots 1, 3, 4 and 5, produces $1 + 4 + 8 + 16 = 29 = 0b00011101$, the cell for ঐ. Because the mask is a set of bits, the order in which the digits are written does not matter. The buffer holds 512 cells, that is 512 bytes, which is ample for any message a user will type into the web form.



brailleBuffer[] — one byte per cell, 512 cells maximum

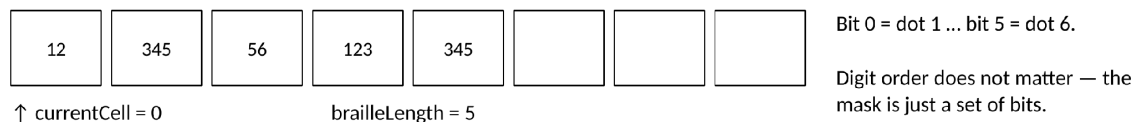


Figure 19: Step 2 — a dot list is converted into a one-byte mask and appended to the cell buffer.

Step 3 — Rule dispatch.

translateBangla() walks the decoded string and tests an ordered list of rules at each position. The first rule that matches wins, consumes the characters it needs and advances the index past them. The order matters: specific rules must be tested before general ones, so numbers and conjuncts are checked before plain letters, otherwise the leading consonant of a cluster would be emitted on its own before the prefix cell could be generated.

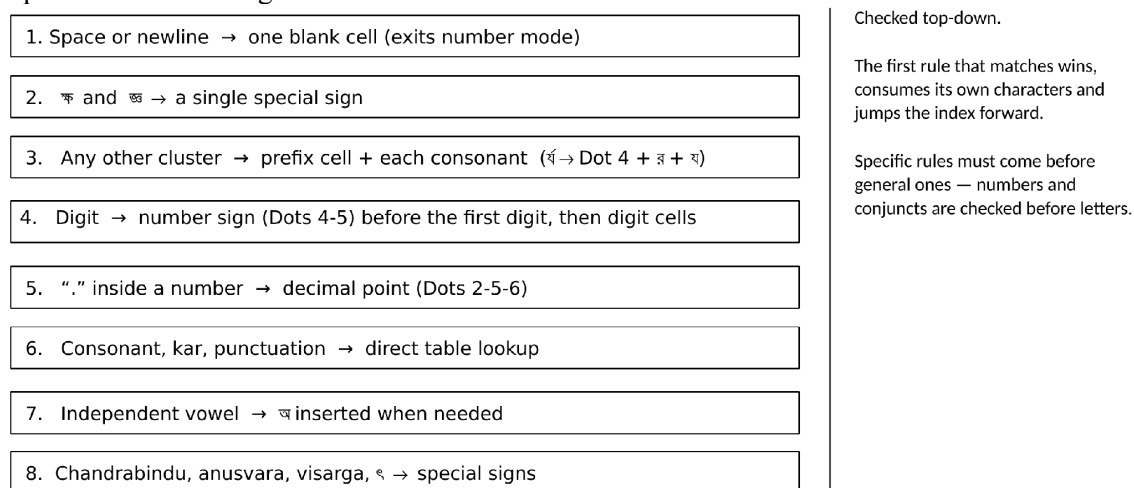


Figure 20: Step 3 — the ordered rule set applied at each position of the input string.

Step 4 — Driving the cell.

showBrailleCell() takes the byte at the current index and writes its six bits to the six GPIO pins, so exactly the dots that belong to that cell are raised. Figure 21 works two examples through the whole pipeline.

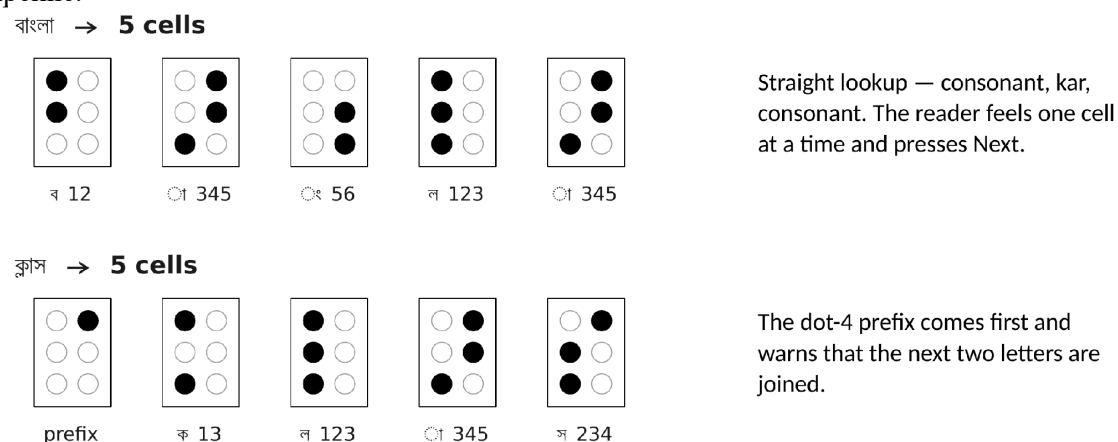


Figure 21: Worked examples: বাংলা is a straight lookup of five cells, while ক্লাস needs the dot-4 conjunct prefix before the joined consonants.

3.3 Circuit Diagram

The complete schematic of the system was captured in Cirkuit Designer [14] and is reproduced in Figure 22. It shows the ESP32 at the centre with the OLED and PCF8575 on the shared I²C bus, the nine pushbuttons on the expander ports, the DFPlayer Mini and speaker on the UART, and the six identical MOSFET driver stages with their flyback diodes feeding the six solenoids of the Braille cell. The 5 V rail from the buck converter supplies the coils and the DFPlayer; the ESP32 and the logic share the same ground reference.

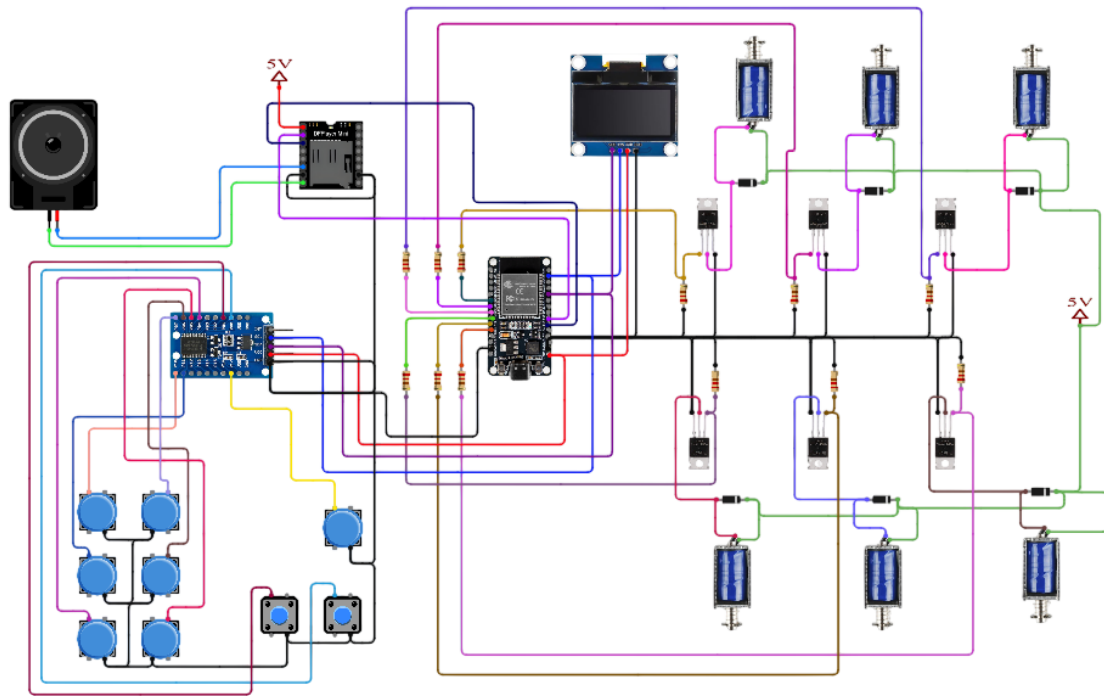


Figure 22: Complete circuit schematic of the bidirectional Bangla Braille translation system, drawn in Cirkuit Designer [14].

3.4 Simulation Model

No simulation model was developed for this project. The system is predominantly a digital, firmware-driven design: the behaviour that determines whether it works is the Bangla Braille rule set and the peripheral timing, neither of which a circuit simulator can predict. The only analogue block, the solenoid driver of Section 3.2.8, is a single-transistor low-side switch whose operating point is fixed by equations (3.3) to (3.6) and can be verified by hand.

Cirkuit Designer [14] was used for schematic capture and connectivity checking rather than for time-domain simulation, and correctness was established experimentally instead: each subsystem — the OLED, the PCF8575 keyboard, the DFPlayer and one solenoid channel — was built and proven on a breadboard before being merged into the single firmware and the final veroboard assembly. Those tests are reported in Section 5.3.

3.5 CAD/Hardware Design

A chassis was modelled in three dimensions to house the electronics and to fix the geometry that the user actually touches. The top face carries the six-dot Braille cell aperture, the six dot keys together with the MODE, ENTER/NEXT and BACKSPACE/PREVIOUS buttons, the OLED window and the speaker grille; the body encloses the veroboard assembly, the buck converter and the battery. The spacing of the solenoid plungers follows the standard Braille cell geometry so that a trained reader's

fingertip finds the dots where it expects them. The 3D model was outsourced under the team's design guidelines.

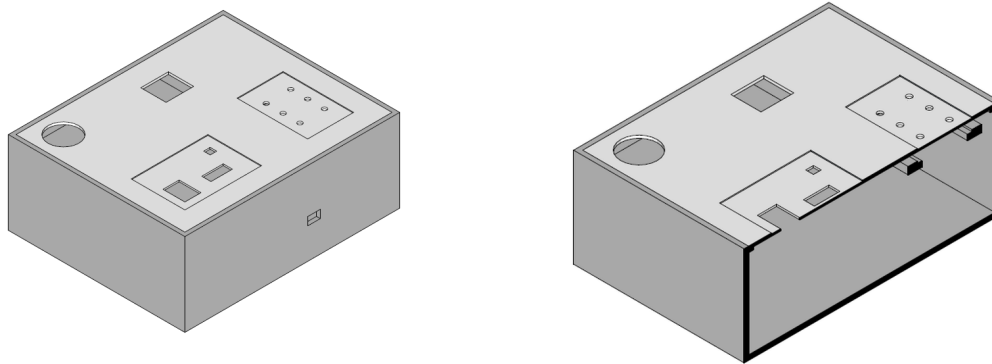


Figure 23: 3D CAD model of the chassis: (left) closed view showing the Braille cell aperture, keypad, display window and speaker opening; (right) cut-away view showing the internal volume for the electronics.

3.6 Veroboard Implementation

No printed circuit board was fabricated for this prototype. The complete circuit was instead implemented on veroboard, with the subsystems distributed over several small boards so that each could be built, tested and replaced independently: one board for the OLED, one for the DFPlayer and speaker, one carrying the PCF8575 expander and the nine pushbuttons, and one carrying the six MOSFET driver stages with the solenoid array mounted above it. Veroboard was chosen because it allowed the design to be corrected during integration; several driver connections and the button port assignment changed after the first tests, which a fabricated board would not have permitted within the project timeline. A proper PCB layout is identified as future work in Section 9.

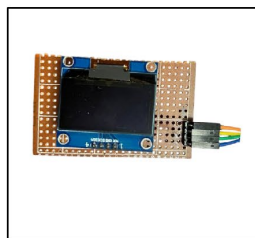


Fig. OLED Connection

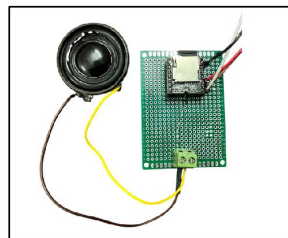


Fig. Speaker Connection

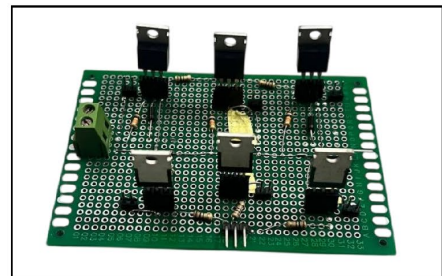


Fig. Solenoid Driving Circuit

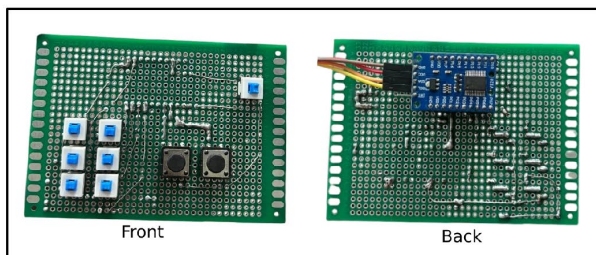


Fig. PCF8575 Connection

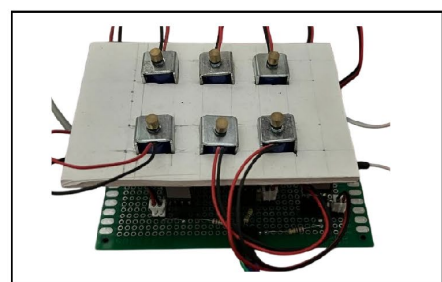


Fig. Solenoid Setup

Figure 24: Circuit implementation on veroboard: OLED board, DFPlayer and speaker board, PCF8575 and pushbutton board (front and back), the six-channel MOSFET driver board, and the solenoid array mounted over it.

4 Implementation

4.1 Description

4.1.1. Building the Input Circuit:

To implement the input circuit, we used six latching push buttons to capture the six-dot Braille pattern. Two additional non-latching push buttons are used for Enter and Backspace. The Enter button confirms the currently selected Braille pattern and registers it as a single character, while the Backspace button removes the previously entered character in case of a mistake. A separate mode-selection button determines the operating mode: when pressed, the device operates in Input Mode, and when released, it operates in Output Mode. The Enter button is used as the Next button in reading mode (which reads the next letter and sends it to output) and the Backspace button as the Previous button (which navigates to the previous letters).

We used an I²C GPIO expander (PCF8575) to read the button inputs, reducing the number of ESP32 GPIOs required for the input interface and avoiding the use of GPIOs with boot-time strapping functions. This provides greater flexibility in GPIO allocation and minimizes the possibility of the external button circuitry interfering with the ESP32's boot configuration. The PCF8575 communicates with the ESP32 through the existing I²C bus shared with the OLED, requiring only the SDA and SCL connections to the ESP32 (excluding VCC and GND). Thus, both devices can efficiently share the same two-wire I²C bus.

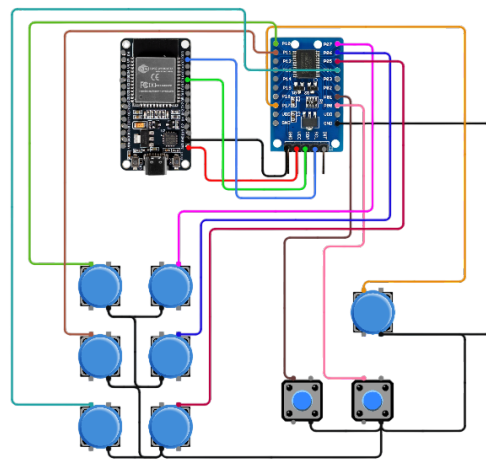


Fig 25: Schematic for input circuit

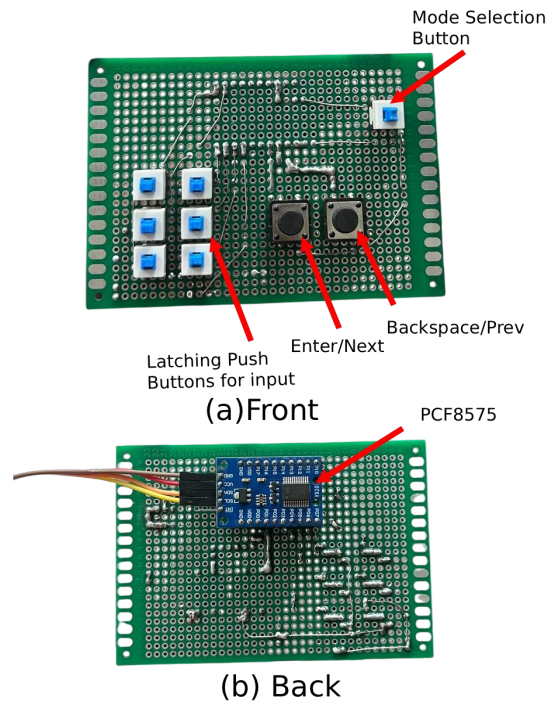


Fig 26: Circuit Implementation of the Input

4.1.2. Speaker Connection:

The audio output is implemented using a DFPlayer Mini module, which communicates with the ESP32 through UART serial communication. The ESP32 sends commands to the DFPlayer to select and play the required audio file corresponding to the detected Braille character. The DFPlayer Mini decodes the stored audio files and outputs the audio signal to the connected speaker, providing audible feedback to the user. Note that the Speaker is only used in the case of Braille Input to aid the blind person with auditory feedback so they can validate the letters they are typing.

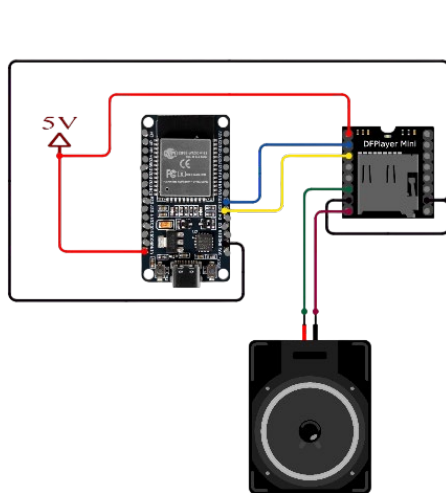


Fig 27: Schematic for Connecting the Speaker

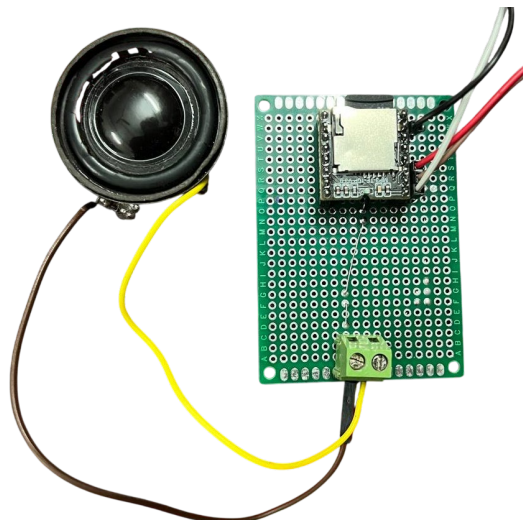


Fig 28: Circuit Implementation of

The Rx,Tx GND and VCC pins of the schematic is different than the connection in our circuit and

that's because the DF mini has different versions. The plus side of the speaker is connected to SPK+ and the minus side is connected to SPK- pins.

The DFPlayer Mini can only play audio files that have been pre-recorded and stored on its memory card; it does not generate speech or convert text into audio. Therefore, we had to create and store individual audio files for the required characters and establish a mapping between each character and its corresponding audio file.

4.1.3. Building the Output Circuit:

For implementing the output side, we used six push-pull solenoids that are driven by 6 MOSFETs. We do not control the Solenoids directly by the ESP32 5V because it is unable to supply the huge current required by the solenoids. Instead, we drive 6 MOSFET gates by the 3V from ESP GPIO output pins, and we connect a 5 Volt DC output from a LiPO+Buck Converter to the MOSFET's drain that supplies the solenoids their desired current. All the 6 solenoids are connected in parallel with the 5V source which can simultaneously drive them with their required current rating. After decoding the letters from the text sent over Wifi hosted webpage, the gating signals are sent to the correct GPIO pins corresponding to the correct dot positions and the MOSFETs, in turn drives the solenoids that raise those particular dots.

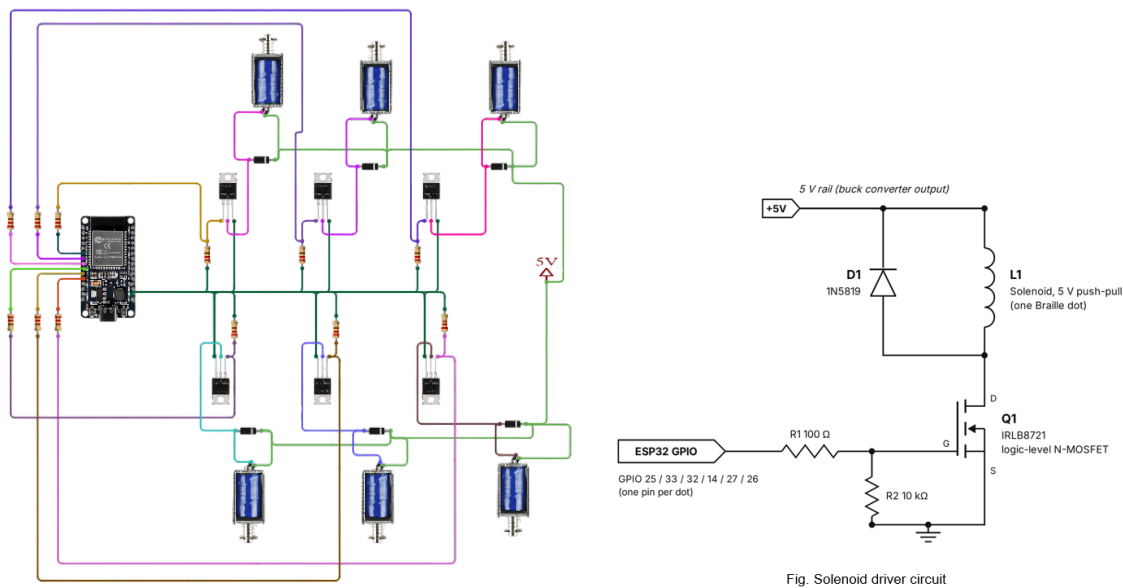


Fig 29: Schematic for the Solenoid Driving Circuit

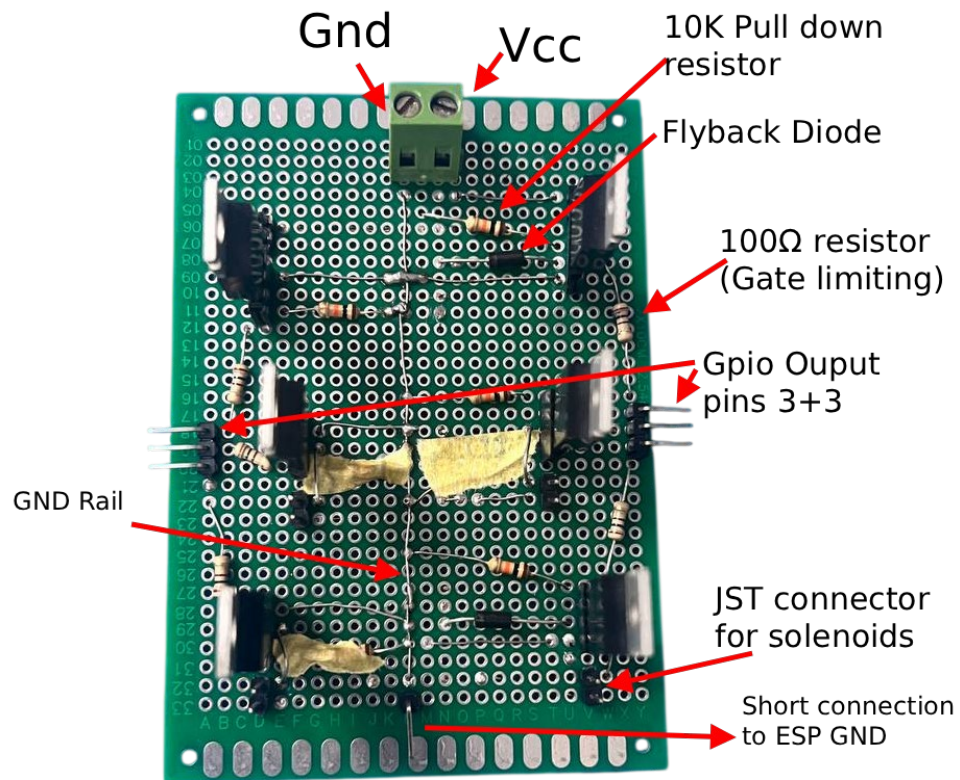


Fig 30: Implementation of the Output Driver Circuit

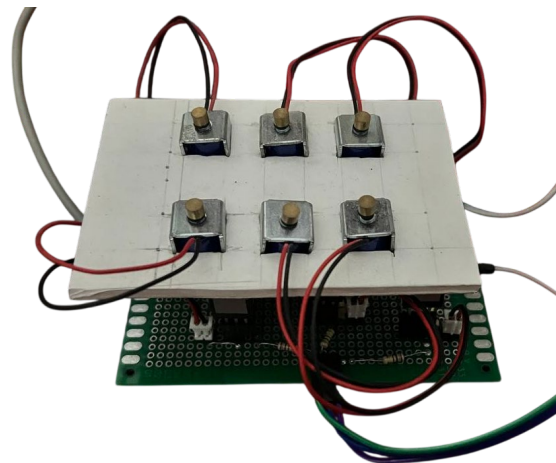
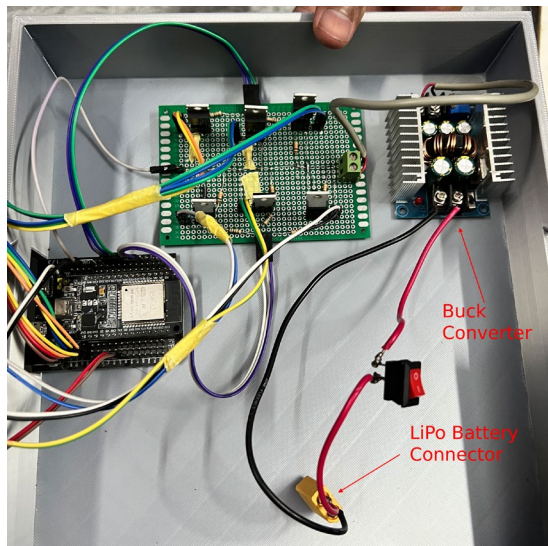


Fig 31: (Left) Connection of the whole output circuit with the power source and ESP32
(Right) After connecting to the solenoids

4.1.4. The OLED Display:

The OLED display is used for a trainer or some assisting person who is helping the blind person to navigate the device. It shows both the text given as input and also the output text that is sent over the hosted webpage. This works with I2C protocol and only has 4 pins VCC,GND,SCL,SDA. The I2C bus is shared with the MUX and causes no vital issue.

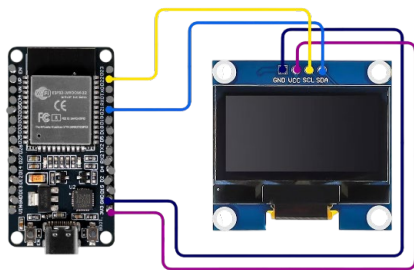


Fig 32: Schematic of the OLED

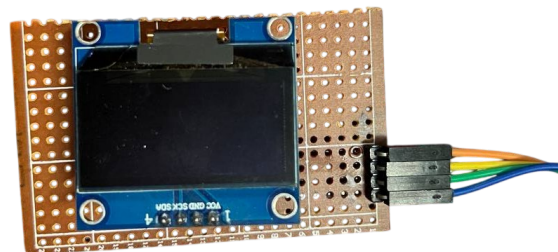


Fig 33: Implementation on the veroboard

4.1.5. Integrating the Complete System:

A custom 3D-printed casing is used to assemble the separate components into a cohesive device. The ESP, battery source, I/O expander, and the Buck Converter are kept inside the box.



Fig. 31: Integration of all the components

5 Design Analysis and Evaluation

5.1 Novelty

1. Use of exact Bangla Braille Grammar: Our system incorporates the linguistic structure of Bangla with proper Braille grammar, including vowel signs (কার), conjunct consonants (যুক্তাক্ষর), punctuation, numbers, and even decimals. Instead of defining our own made up rules, we used the exact rules Braille is written and read in Bangla language that will make the user experience more natural.

2. Bidirectional Braille Interaction: The device supports both Braille input and Braille output. Users can enter Braille characters through the six-button input interface, while the device translates Bangla text and represents it physically through the tactile Braille output interface. It is hard to find a device that works both ways, especially when resources are scarce in the Bangla language.

3. Integrated audio feedback: Our DFPlayer+speaker can provide audio feedback for Braille **input**. So a user entering a Braille pattern can hear the corresponding character and validate the input he/she just entered.

4. Standalone Operation: The device performs the Bangla-to-Braille translation and controls the tactile and audio outputs directly through the ESP32, allowing the system to operate independently without requiring a computer for processing.

5.2 Design Considerations (PO(c))

5.2.1 Considerations for public health and safety

- The device was designed with safe, low-voltage electronic components to minimize the risk of electrical hazards during operation. Also we kept everything inside a plastic box which ensures the user does not accidentally touch any electrical components that can cause short circuit.
- The tactile solenoids are controlled through appropriate driver circuits rather than directly from the ESP32 GPIO pins, preventing excessive current from being drawn from the microcontroller and causing unwanted harm. Gate current limiting resistors are used in the mosfets and the current and ratings are matched for safe operation. Flyback diodes are used so that while discharging the solenoids don't harm the circuit.
- The solenoids are not held in charged position for more than 3 seconds at a time ensuring the coils do not heat up excessively. Instead we “flash” each letter for 3 seconds at a time give it a gap and then repeat again.

5.2.2 Considerations to environment

The prototype uses reusable electronic components such as the ESP32, OLED display, PCF8575 module, DFPlayer Mini, and solenoids. The system is also designed around a rechargeable LiPo battery, reducing the need for disposable batteries. However, the solenoids consume relatively high electrical power. Future improvements should therefore focus on reducing power consumption and improving the efficiency of the tactile Braille mechanism. Electronic components and batteries should also be disposed of properly at the end of their useful life.

5.2.3 Considerations to cultural and societal needs

- The device is specifically designed to support Bangla Braille, addressing the needs of Bangla-speaking visually impaired users. This device works as a all in one solution for both bangla braille typing and reading from text file dynamically.
- The translation system considers Bangla-specific linguistic structures, including vowel signs (কার), conjunct consonants (যুক্তাক্ষর), punctuations and numbers. Thus it adds to the cultural significance
- Audio feedback is also incorporated to improve accessibility and assist users in identifying and verifying the entered Braille characters

5.3 Limitations of Tools (PO(e))

- **Single Braille Cell:** Only one six-dot Braille cell is used; therefore, long text must be read one character at a time.
- **Limited Memory:** Only a fixed amount of Braille data can be stored at a time.
- **Pre-recorded Audio:** Audio feedback depends on MP3 files stored on the microSD card.
- **No Automatic Voice Generation:** The system cannot automatically generate speech for complete words or sentences.
- **Local Wi-Fi Access:** The webpage works only through the local Wi-Fi network created by the ESP32.

- **Limited Character Support:** Some punctuation marks and complex symbols are not supported in the current system.
- **High Power Consumption:** The solenoids require relatively high current during operation.
- **Solenoid Heating and Speed:** The solenoids are comparatively slow and may generate heat during prolonged operation.

5.4 Impact Assessment (PO(f))

5.4.1 Assessment of Societal and Cultural Issues

The proposed system may help improve communication between Bangla Braille users and people who do not know Braille. Its bidirectional operation can support communication in educational and everyday environments. The system may also assist in Bangla Braille education and practice. The Braille input interface can additionally serve as a basic typing mechanism for visually impaired users. Because the device specifically processes Bangla Braille, it addresses a language-specific accessibility requirement rather than relying only on systems designed for other languages.

5.4.2 Assessment of Health and Safety Issues

The tactile Braille output provides information without requiring the user to depend on a visual interface. This is particularly relevant for visually impaired users. Electrical and mechanical safety were also considered in the hardware design. The solenoids are controlled through driver circuits rather than directly from the ESP32, and protective components are used in the switching circuit. Nevertheless, solenoid heating and high current consumption remain important considerations for prolonged operation and should be further improved in future designs.

5.4.3 Assessment of Legal Issues

The Bangla text entered by the user is processed locally by the ESP32 web server. Therefore, the basic operation of the prototype does not require the entered text to be sent to an external cloud server. The developed prototype is intended as an educational assistive system and should not be presented as a certified commercial accessibility or medical device without appropriate testing, validation, and certification.

5.5 Sustainability Evaluation (PO(g))

The proposed system has potential for further development as a low-cost assistive technology. The ESP32-based architecture allows several functions—including processing, Wi-Fi communication, webpage hosting, and hardware control—to be

integrated into a single controller. The sustainability of the design can be improved by replacing the Veroboard implementation with a compact custom PCB, reducing the power consumption of the tactile output mechanism, and improving the mechanical design of the Braille cell. The use of a rechargeable power source can also reduce dependence on disposable batteries. A major concern is the relatively high power requirement and heating of the solenoids. Therefore, future versions should use a more energy-efficient tactile actuation mechanism or optimized solenoid control to improve long-term sustainability.

5.6 Ethical Issues (PO(h))

Throughout the development of the device, we maintained professional integrity by accurately documenting the design, implementation, testing, and limitations of the system without overstating its capabilities. We followed responsible engineering practices by prioritizing user safety during circuit design and testing, using components and software in accordance with their intended requirements, and properly acknowledging externally sourced materials. Privacy was also considered when handling uploaded text and user inputs, ensuring that the system did not unnecessarily collect or retain personal information.

6 Reflection on Individual and Team work (PO(i))

6.1 Individual Contribution of Each Member

Member	Student ID	Main Contribution
Mushfiquir Rahman	2106181	Bangla text processing(Conjunct, Punctuation, numbers, special characters), Text to Braille processing, Circuit soldering, Total system integration, Documentation
Chayan Paul	2106182	Braille to audio processing, DFPlayer Bug fix, Solenoid circuit operation, Logistics, System integration, Documentation
Shamin Yeaser	2106183	Bangla text processing(Kar, Conjunct), DFPlayer testing, Display setup for Bangla text, System integration, Logistics, Documentation
Kayes Sami Malitha	2106184	Webpage hosting, Circuit soldering, audio file conversion, Logistics, System integration, Documentation

6.2 Mode of Teamwork

The project was developed using a collaborative approach. At the beginning, the complete system was divided into several smaller subsystems, such as Bangla text processing, Braille input processing, audio feedback, OLED display, webpage hosting, solenoid control, and hardware implementation.

Individual members initially worked on their assigned tasks. After completing the individual modules, the modules were gradually combined into a single ESP32-based system. During integration, all members participated in testing, debugging, circuit checking, and modification of the firmware.

Regular discussions among the members were important because most parts of the project were dependent on each other. For example, the Bangla processing algorithm had to work correctly with the Braille lookup table, while the generated Braille pattern had to match the

physical output produced by the six solenoids. Similarly, Braille input had to be coordinated with the OLED and audio feedback systems.

Therefore, the project required both individual responsibility and continuous teamwork.

6.3 Diversity Statement of Team

The project team consisted of four members with different technical responsibilities and problem-solving roles. This diversity of tasks helped the team address different aspects of the system, including embedded programming, Bangla Unicode processing, hardware implementation, audio processing, web development, and documentation.

Team members shared their individual findings with the rest of the group during system integration. Decisions related to hardware connections, software logic, debugging, and presentation were made through group discussion. This collaborative environment helped combine different technical skills into a complete working prototype.

6.4 Log Book of Project Implementation

Date	Milestone achieved	Individual Role	Team Role	Comments
Week 6	Literature review on Bangla Braille rules and Bangla text processing rules	2106183 reviewed Braille rules and 2106181 reviewed Bangla text processing rules	Team discussed translation rules	Translation pipeline approach finalized
Week 6	Bangla text rendering on OLED	2106181 reviewed BanglaText library and 2106182 and 183 implemented line processing	Team verified Bangla text rendering cases	Bangla text rendering approach finalized
Week 7	Webpage hosted using ESP32	2106184 implemented the webpage and received Bangla text over wifi	Team implemented text validation rule	Text entry over wifi finalized
Week 7	Reading Braille dots from 6 push buttons	2106183 implemented algorithm for Braille reading and implemented normal character processing	Team verified operation	Braille pattern reading algorithm finalized
Week 8	Implementing Kar and conjunct	2106183 and 181 implemented kar and conjunct operation.	Team verified the operation	Braille processing rule added

Week 8	Implemented ঙ and ঞ	181 modified the algorithm to include the characters. 182 and 184 performed debugging.	Team verified operation	Braille rule enhanced
Week 9	Solenoid operation circuit verified	182 implemented the solenoid driver circuit	Team verified the operation	Solenoid operated
Week 9	I/O expander operated	181 implemented the I/O expander. 184 helped	Team verified pushbutton operation	I/O expander operated
Week 10	Bangla to Braille translation pipeline finalized	181 implemented the algorithm and 183 added the next/previous feature.	Team verified operation	Bangla to Braille translation verified on LED for simplicity
Week 11	Speaker operated using DFPlayer	182 and 183 implemented the circuit and code	Team verified operation	Speaker operated
Week 11	Circuit implemented on veroboard	184 and 181 soldered the circuits on the veroboard and 182 helped	Team discussed the system architecture	Circuit implementation completed
Week 12	Added numbers and punctuation	181 added numbers and punctuation	Team performed debugging	Number mode implemented
Week 13	Final debugging and code verification	182 and 183 performed final proofreading	Team verified operation	Code finalized
Week 14	3D model assembly	183 managed the model design, 184 and 181 assembled the system	Team verified correct operation	Project completed
Week 14	Documentation completed	All performed	Team organized the project documents	Report, presentation, poster prepared

7 Communication to External Stakeholders (PO(j))

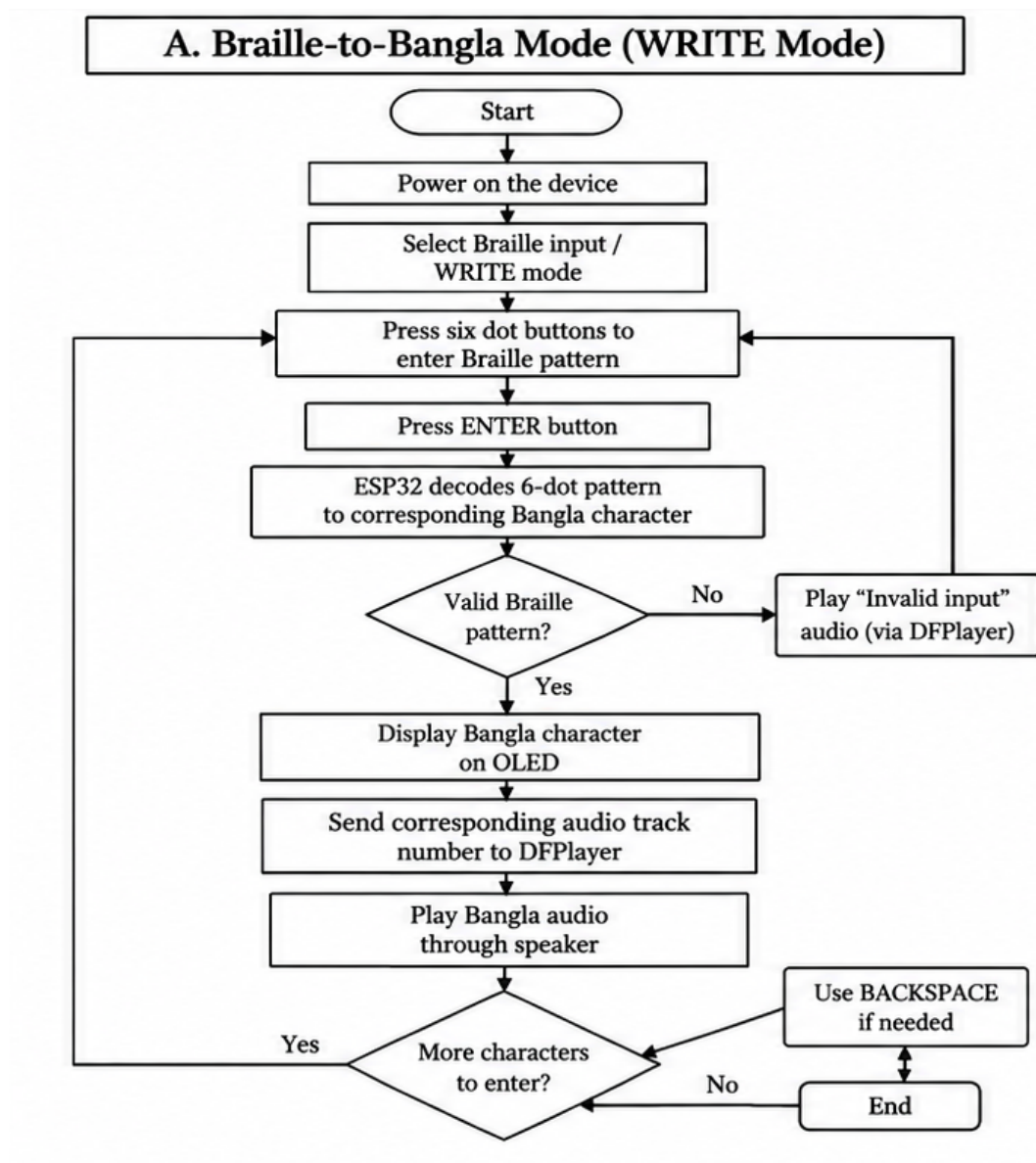
7.1 Executive Summary

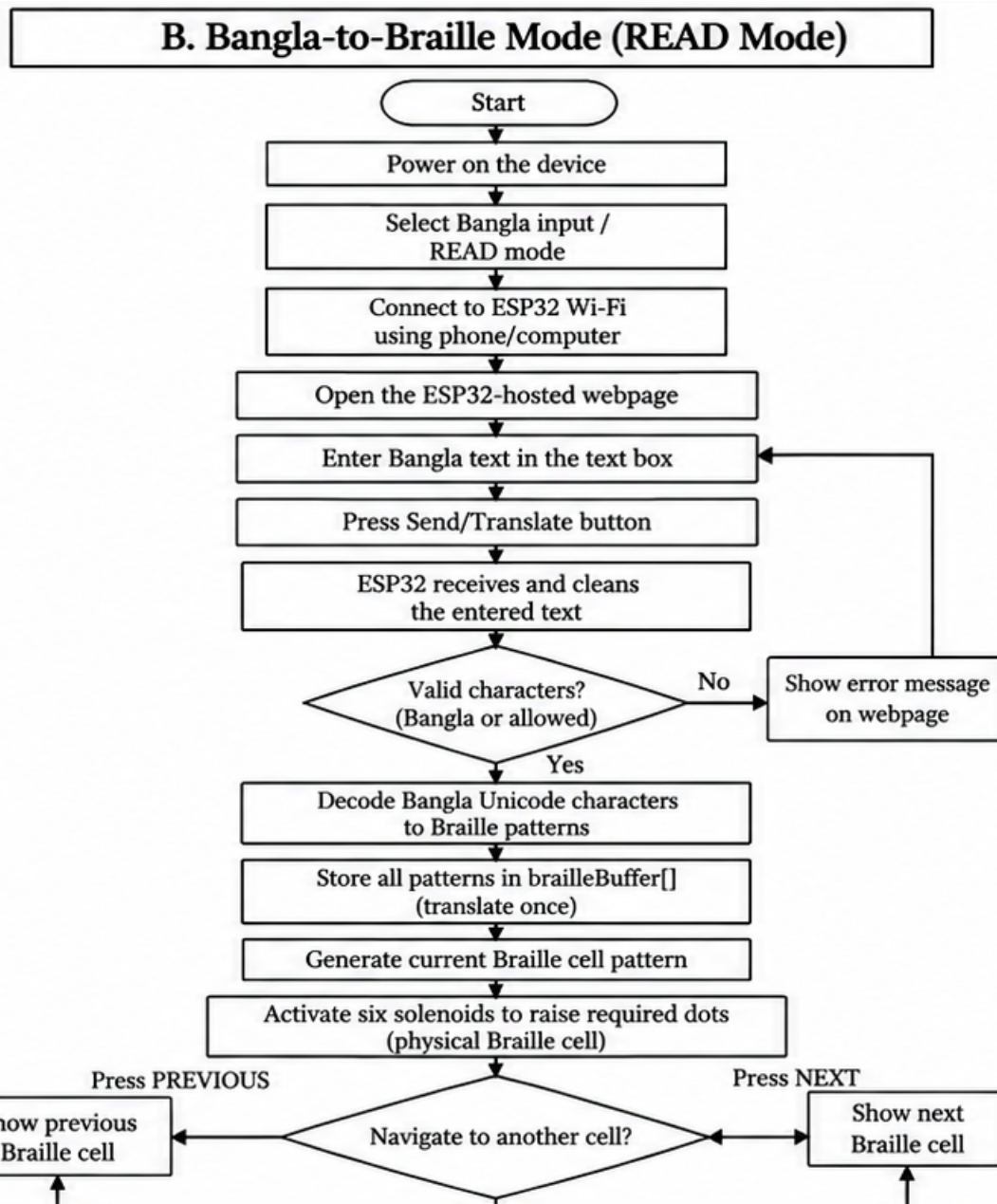
The Bidirectional Bangla Braille Translation System is an ESP32-based assistive device designed to improve communication between Bangla Braille users and people who do not understand Braille. The device supports communication in both directions. A user can enter a six-dot Braille pattern and obtain the corresponding Bangla character through an OLED display and audio feedback. In the opposite direction, Bangla text can be entered through a locally hosted webpage and converted into tactile Braille using a six-solenoid refreshable Braille cell. The prototype combines tactile, visual, and audio feedback in a low-cost

embedded platform and demonstrates a practical approach to Bangla Braille communication.

7.2 User Manual

The device has two main operating modes: Braille-to-Bangla and Bangla-to-Braille.





7.3 Github Link

<https://github.com/intmushfiq/Bidirectional-Bangla-Braille-Device>

7.4 YouTube Link

<https://youtu.be/4rY5ggd-0BE>

8 Project Management and Cost Analysis (PO(k))

8.1 Bill of Material

Item	Qty	Unit(TK)	Total(TK)
ESP32 DevKit (30-pin)	1	480	480
SH1106 OLED 1.3" (I ² C)	1	600	600
PCF8575 I/O expander module	1	350	350
DFPlayer Mini	1	240	240
32 GB microSD	1	650	650
4 Ω 3 W speaker	1	280	280
5 V push-pull solenoid	6	250	1500
IRLB8721 MOSFET	6	150	900
1N5819 diode, resistors	—	—	50
Push buttons (6 latching, 3 other)	9	10	90
LiPo Battery + Buck Converter	1	2200	2200
Veroboard, wires, enclosure	—	—	500
Enclosure (3D Printing)	1	3000	3000
Prototype total			10,840

8.2 Calculation of Per Unit Cost of Prototype

The approximate cost of one prototype is 10840 Tk.

8.3 Calculation of Per Unit Cost of Mass-Produced Unit

The per-unit cost can be reduced when the device is produced on a larger scale. This can be achieved by purchasing components in bulk, using a custom PCB instead of separate modules and Veroboard, and optimizing the power supply and hardware design.

Based on these considerations, the estimated cost of one mass-produced unit is approximately BDT 6,500, which represents a cost reduction of approximately 35–40% compared with the prototype.

9 Future Work (PO(I))

- Add multiple Braille cells for faster reading.
- Design PCB layout for a compact device.
- Add Bangla text-to-speech instead of recorded audio.
- Support more Bangla punctuation marks and symbols.
- Add English or other language processing capability.
- Improve the Braille cell design.
- Reduce power use and heating.
- Test the device with visually impaired users and Braille teachers

10 References

- [1] American Foundation for the Blind, “Refreshable Braille Displays.” [Online]. Available: afb.org/blindness-and-low-vision/using-technology/assistive-technology-products/refreshable-braille
- [2] C. Chong, “The Orbit Reader 20: The Most Inexpensive Braille Display,” Braille Monitor, NFB, Oct. 2017.
- [3] “ব্রেইল পদ্ধতি,” উইকিপিডিয়া (Bangla Wikipedia). [Online]. Available: https://bn.wikipedia.org/wiki/ব্রেইল_পদ্ধতি (accessed Sep. 25, 2026).
- [4] Shop4specials Pvt. Ltd., “Humanware Refreshable Braille Display Brailliant BI 40X with Audio,” IndiaMART. [Online]. Available: <https://www.indiamart.com/proddetail/humanware-refreshable-braille-display-brailliant-bi-40x-with-audio-27011004962.html> (accessed Sep. 25, 2026).
- [5] M. S. Rahman, M. M. Rahman and M. Kaykobad, “Bangla Braille Adaptation,” in Technical Challenges and Design Issues in Bangla Language Processing, IGI Global, 2013, ch. 2, pp. 17–34.

- [6] World Braille Usage, 3rd ed. Watertown, MA, USA: Perkins; Toronto, ON, Canada: International Council on English Braille; Washington, DC, USA: National Library Service for the Blind and Physically Handicapped, Library of Congress, 2013, “Bangladesh,” pp. 9–10. ISBN 978-0-8444-9564-4.
- [7] M. Kamal, A. A. Ali, M. A. H. Khan and M. Shoyaib, “Braille to Text Translation for Bengali Language: A Geometric Approach,” arXiv:2012.01494, 2020.
- [8] U. W. Tabassum, S. N. Soumma, T. Tasfia and A. S. Gupta, “Design & Fabrication of a Cost-effective, Refreshable Mechanical Braille Display for Visually Impaired Students in Bangladesh,” SSRN 4862819.
- [9] Science Buddies, “Build a Refreshable Braille Display with Arduino.” [Online]. Available: sciencebuddies.org
- [10] Espressif Systems, ESP32 Series Datasheet; Texas Instruments, PCF8575 datasheet; DFRobot, DFPlayer Mini wiki; Infineon, IRLB8721 datasheet.
- [11] M. Mazid, BanglaText — Bangla text rendering library for microcontrollers. [Online]. Available: github.com/mamunul/BanglaText
- [12] O. Kraus, U8g2 graphics library for monochrome displays. [Online]. Available: github.com/olikraus/u8g2
- [13] M. Zaman (lifeparticle), “Bengali-Alphabet,” GitHub repository, MIT License. [Online]. Available: <https://github.com/lifeparticle/Bengali-Alphabet> (accessed Sep. 25, 2026).
- [14] Circuit Designer, online circuit design and simulation tool. [Online]. Available: <https://www.circuitdesigner.com> (accessed Sep. 25, 2026).